Lab Report No 10
Implementation of Image compression using Quantization

Digital Image Processing

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

13th Dec,2024

Department of Computer Engineering,

HITEC University, Taxila

Task No 1:

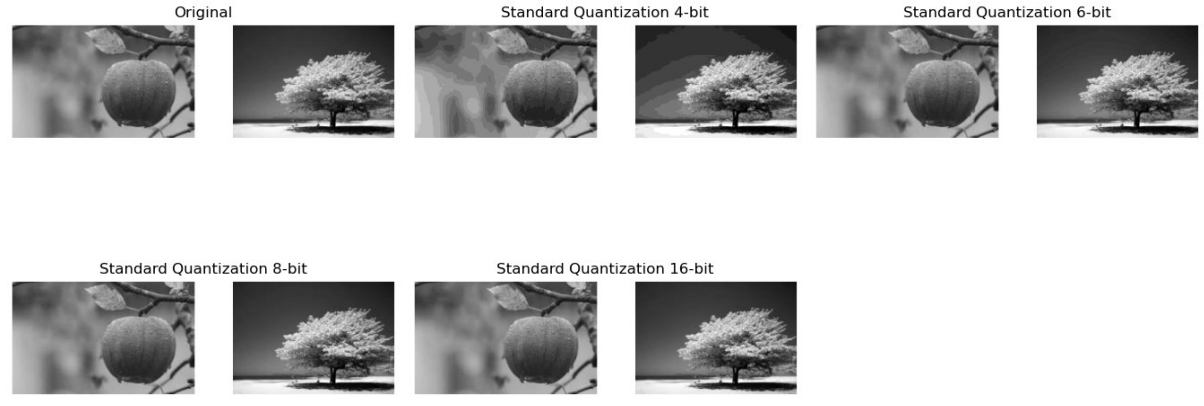
Solution:

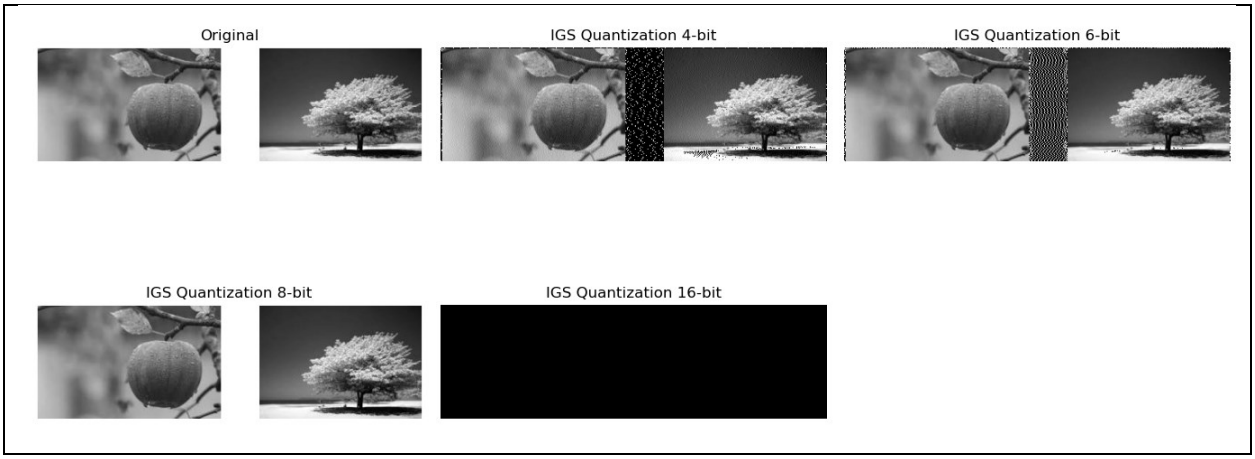
Brief description (3-5 lines)
The code

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
def standard_quantization(image, bits):
    levels = 2 ** bits
    quantized_image = np.floor(image / (256 / levels)) * (256 / levels)
    return quantized_image.astype(np.uint8)
def igs_quantization(image, bits):
    levels = 2 ** bits
    quantized_image = np.zeros_like(image, dtype=np.uint8)
    error = 0
    for row in range(image.shape[0]):
        for col in range(image.shape[1]):
            pixel_value = image[row, col]
            new_value = (pixel_value + error) // (256 // levels) * (256 //
levels)
            error = pixel_value + error - new_value
            quantized_image[row, col] = new_value
    return quantized_image
def plot_images(original, quantized_images, title_prefix):
    plt.figure(figsize=(10, 8))
    plt.subplot(2, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title("Original")
    plt.axis('off')
    for i, (bits, q_image) in enumerate(quantized_images):
        plt.subplot(2, 3, i + 2)
        plt.imshow(q_image, cmap='gray')
```

```
        plt.title(f"{title_prefix} {bits}-bit")
        plt.axis('off')
    plt.tight_layout()
    plt.show()
# Load the grayscale image
image_path = ('APPLEANDTREE.jpg')
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if image is None:
    raise ValueError("Image not found or not readable.")
# Perform quantization
bit_depths = [4, 6, 8, 16]
standard_quantized_images = [(bits, standard_quantization(image,
bits)) for bits in bit_depths]
igs_quantized_images = [(bits, igs_quantization(image, bits)) for bits
in bit_depths]
# Plot results
plot_images(image, standard_quantized_images, "Standard
Quantization")
plot_images(image, igs_quantized_images, "IGS Quantization")
```

The results (Screenshot)





Lab Task No 2:

Solution:

Brief description (3-5 lines)
COLOR SCALING: **Quantization in Standard:** Linear mapping of color values spread evenly into uniform color space. **Quantization in IGS:** Non-linear scaling better preserve perceptual color relationship than mere uniform sampling. **Standard method:** The step sizes between quantization levels are equal to that which may cause color bandings. **IGS method:** **Adapted step sizes:** More levels are concentrated perceptually important areas. Perceptual priority in IGS- finer representation for high sensitivity areas.
The code

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image 	
image = cv2.imread('COLORSCALING.jpg') # Replace 'image.jpg' with your image path
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert from BGR to RGB

```
# Function to perform image quantization
def quantize_image(image, levels):
    step = 256 // levels
    quantized = (image // step) * step + step // 2
    return np.clip(quantized, 0, 255).astype(np.uint8)
# Quantize the image with a given number of levels
quantized_image_4 = quantize_image(image, levels=4)
quantized_image_8 = quantize_image(image, levels=8)
quantized_image_6 = quantize_image(image, levels=6)
quantized_image_16 = quantize_image(image, levels=16)
# Function to perform intensity slicing
def intensity_slice(channel, min_val, max_val):
    sliced = np.zeros_like(channel)
    sliced[(channel >= min_val) & (channel <= max_val)] =
channel[(channel >= min_val) & (channel <= max_val)]
    return sliced
# Slice intensity ranges for each channel
red_channel, green_channel, blue_channel = image[:, :, 0], image[:, :,
1], image[:, :, 2]
red_sliced = intensity_slice(red_channel, 100, 200)
green_sliced = intensity_slice(green_channel, 50, 150)
blue_sliced = intensity_slice(blue_channel, 0, 100)
# Quantize the sliced channels
red_quantized_4 = quantize_image(red_sliced, levels=4)
green_quantized_4 = quantize_image(green_sliced, levels=4)
blue_quantized_4 = quantize_image(blue_sliced, levels=4)
red_quantized_8 = quantize_image(red_sliced, levels=8)
green_quantized_8 = quantize_image(green_sliced, levels=8)
blue_quantized_8 = quantize_image(blue_sliced, levels=8)
red_quantized_6 = quantize_image(red_sliced, levels=6)
green_quantized_6 = quantize_image(green_sliced, levels=6)
blue_quantized_6 = quantize_image(blue_sliced, levels=6)
red_quantized_16 = quantize_image(red_sliced, levels=16)
green_quantized_16 = quantize_image(green_sliced, levels=16)
blue_quantized_16 = quantize_image(blue_sliced, levels=16)
```

```
# Display the results
fig, axs = plt.subplots(5, 4, figsize=(20, 20))
# Original image and quantized images
axs[0, 0].imshow(image)
axs[0, 0].set_title("Original Image")
axs[0, 0].axis('off')
axs[0, 1].imshow(quantized_image_4)
axs[0, 1].set_title("Quantized (4 levels)")
axs[0, 1].axis('off')
axs[0, 2].imshow(quantized_image_8)
axs[0, 2].set_title("Quantized (8 levels)")
axs[0, 2].axis('off')
axs[0, 3].imshow(quantized_image_6)
axs[0, 3].set_title("Quantized (6 levels)")
axs[0, 3].axis('off')
axs[1, 0].imshow(quantized_image_16)
axs[1, 0].set_title("Quantized (16 levels)")
axs[1, 0].axis('off')
# Red channel slicing and quantization
axs[1, 1].imshow(red_sliced, cmap='Reds')
axs[1, 1].set_title("Red Channel Sliced")
axs[1, 1].axis('off')
axs[1, 2].imshow(red_quantized_4, cmap='Reds')
axs[1, 2].set_title("Red Channel Quantized (4 levels)")
axs[1, 2].axis('off')
axs[1, 3].imshow(red_quantized_8, cmap='Reds')
axs[1, 3].set_title("Red Channel Quantized (8 levels)")
axs[1, 3].axis('off')
axs[2, 0].imshow(red_quantized_6, cmap='Reds')
axs[2, 0].set_title("Red Channel Quantized (6 levels)")
axs[2, 0].axis('off')
axs[2, 1].imshow(red_quantized_16, cmap='Reds')
axs[2, 1].set_title("Red Channel Quantized (16 levels)")
axs[2, 1].axis('off')
# Green channel slicing and quantization
```

```
axs[2, 2].imshow(green_sliced, cmap='Greens')
axs[2, 2].set_title("Green Channel Sliced")
axs[2, 2].axis('off')
axs[2, 3].imshow(green_quantized_4, cmap='Greens')
axs[2, 3].set_title("Green Channel Quantized (4 levels)")
axs[2, 3].axis('off')
axs[3, 0].imshow(green_quantized_8, cmap='Greens')
axs[3, 0].set_title("Green Channel Quantized (8 levels)")
axs[3, 0].axis('off')
axs[3, 1].imshow(green_quantized_6, cmap='Greens')
axs[3, 1].set_title("Green Channel Quantized (6 levels)")
axs[3, 1].axis('off')
axs[3, 2].imshow(green_quantized_16, cmap='Greens')
axs[3, 2].set_title("Green Channel Quantized (16 levels)")
axs[3, 2].axis('off')
# Blue channel slicing and quantization
axs[3, 3].imshow(blue_sliced, cmap='Blues')
axs[3, 3].set_title("Blue Channel Sliced")
axs[3, 3].axis('off')
axs[4, 0].imshow(blue_quantized_4, cmap='Blues')
axs[4, 0].set_title("Blue Channel Quantized (4 levels)")
axs[4, 0].axis('off')
axs[4, 1].imshow(blue_quantized_8, cmap='Blues')
axs[4, 1].set_title("Blue Channel Quantized (8 levels)")
axs[4, 1].axis('off')
axs[4, 2].imshow(blue_quantized_6, cmap='Blues')
axs[4, 2].set_title("Blue Channel Quantized (6 levels)")
axs[4, 2].axis('off')
axs[4, 3].imshow(blue_quantized_16, cmap='Blues')
axs[4, 3].set_title("Blue Channel Quantized (16 levels)")
axs[4, 3].axis('off')
plt.tight_layout()
plt.show()
import cv2
import numpy as np
```

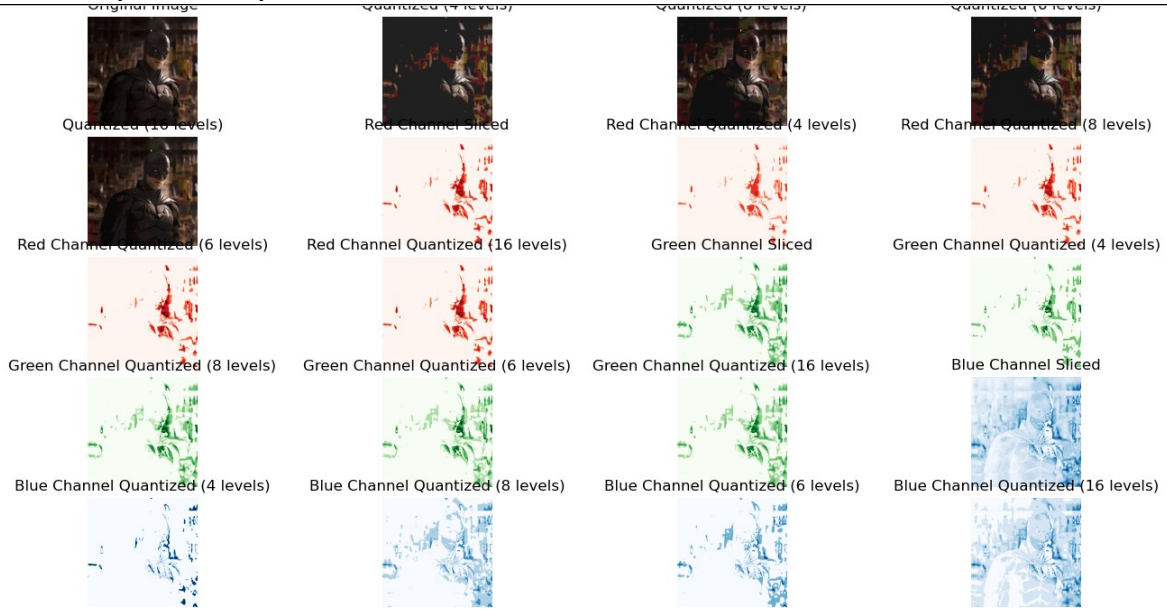
```
import matplotlib.pyplot as plt
# Load the image
image = cv2.imread('COLORSCALING.jpg')  # Replace 'image.jpg'
with your image path
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  # Convert
from BGR to RGB
# Function to perform image quantization
def quantize_image(image, levels):
    step = 256 // levels
    quantized = (image // step) * step + step // 2
    return np.clip(quantized, 0, 255).astype(np.uint8)
# Quantize the image with a given number of levels
quantized_image_4 = quantize_image(image, levels=4)
quantized_image_8 = quantize_image(image, levels=8)
quantized_image_6 = quantize_image(image, levels=6)
quantized_image_16 = quantize_image(image, levels=16)
# Function to perform intensity slicing
def intensity_slice(channel, min_val, max_val):
    sliced = np.zeros_like(channel)
    sliced[(channel >= min_val) & (channel <= max_val)] =
channel[(channel >= min_val) & (channel <= max_val)]
    return sliced
# Slice intensity ranges for each channel
red_channel, green_channel, blue_channel = image[:, :, 0], image[:, :,
1], image[:, :, 2]
red_sliced = intensity_slice(red_channel, 100, 200)
green_sliced = intensity_slice(green_channel, 50, 150)
blue_sliced = intensity_slice(blue_channel, 0, 100)
# Quantize the sliced channels
red_quantized_4 = quantize_image(red_sliced, levels=4)
green_quantized_4 = quantize_image(green_sliced, levels=4)
blue_quantized_4 = quantize_image(blue_sliced, levels=4)
red_quantized_8 = quantize_image(red_sliced, levels=8)
green_quantized_8 = quantize_image(green_sliced, levels=8)
blue_quantized_8 = quantize_image(blue_sliced, levels=8)
```

```
red_quantized_6 = quantize_image(red_sliced, levels=6)
green_quantized_6 = quantize_image(green_sliced, levels=6)
blue_quantized_6 = quantize_image(blue_sliced, levels=6)
red_quantized_16 = quantize_image(red_sliced, levels=16)
green_quantized_16 = quantize_image(green_sliced, levels=16)
blue_quantized_16 = quantize_image(blue_sliced, levels=16)
# Display the results
fig, axs = plt.subplots(5, 4, figsize=(20, 20))
# Original image and quantized images
axs[0, 0].imshow(image)
axs[0, 0].set_title("Original Image")
axs[0, 0].axis('off')
axs[0, 1].imshow(quantized_image_4)
axs[0, 1].set_title("Quantized (4 levels)")
axs[0, 1].axis('off')
axs[0, 2].imshow(quantized_image_8)
axs[0, 2].set_title("Quantized (8 levels)")
axs[0, 2].axis('off')
axs[0, 3].imshow(quantized_image_6)
axs[0, 3].set_title("Quantized (6 levels)")
axs[0, 3].axis('off')
axs[1, 0].imshow(quantized_image_16)
axs[1, 0].set_title("Quantized (16 levels)")
axs[1, 0].axis('off')
# Red channel slicing and quantization
axs[1, 1].imshow(red_sliced, cmap='Reds')
axs[1, 1].set_title("Red Channel Sliced")
axs[1, 1].axis('off')
axs[1, 2].imshow(red_quantized_4, cmap='Reds')
axs[1, 2].set_title("Red Channel Quantized (4 levels)")
axs[1, 2].axis('off')
axs[1, 3].imshow(red_quantized_8, cmap='Reds')
axs[1, 3].set_title("Red Channel Quantized (8 levels)")
axs[1, 3].axis('off')
axs[2, 0].imshow(red_quantized_6, cmap='Reds')
```

```
axs[2, 0].set_title("Red Channel Quantized (6 levels)")
axs[2, 0].axis('off')
axs[2, 1].imshow(red_quantized_16, cmap='Reds')
axs[2, 1].set_title("Red Channel Quantized (16 levels)")
axs[2, 1].axis('off')
# Green channel slicing and quantization
axs[2, 2].imshow(green_sliced, cmap='Greens')
axs[2, 2].set_title("Green Channel Sliced")
axs[2, 2].axis('off')
axs[2, 3].imshow(green_quantized_4, cmap='Greens')
axs[2, 3].set_title("Green Channel Quantized (4 levels)")
axs[2, 3].axis('off')
axs[3, 0].imshow(green_quantized_8, cmap='Greens')
axs[3, 0].set_title("Green Channel Quantized (8 levels)")
axs[3, 0].axis('off')
axs[3, 1].imshow(green_quantized_6, cmap='Greens')
axs[3, 1].set_title("Green Channel Quantized (6 levels)")
axs[3, 1].axis('off')
axs[3, 2].imshow(green_quantized_16, cmap='Greens')
axs[3, 2].set_title("Green Channel Quantized (16 levels)")
axs[3, 2].axis('off')
# Blue channel slicing and quantization
axs[3, 3].imshow(blue_sliced, cmap='Blues')
axs[3, 3].set_title("Blue Channel Sliced")
axs[3, 3].axis('off')
axs[4, 0].imshow(blue_quantized_4, cmap='Blues')
axs[4, 0].set_title("Blue Channel Quantized (4 levels)")
axs[4, 0].axis('off')
axs[4, 1].imshow(blue_quantized_8, cmap='Blues')
axs[4, 1].set_title("Blue Channel Quantized (8 levels)")
axs[4, 1].axis('off')
axs[4, 2].imshow(blue_quantized_6, cmap='Blues')
axs[4, 2].set_title("Blue Channel Quantized (6 levels)")
axs[4, 2].axis('off')
axs[4, 3].imshow(blue_quantized_16, cmap='Blues')
```

```
axs[4, 3].set_title("Blue Channel Quantized (16 levels)")
axs[4, 3].axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Conclusion

In conclusion,the quantization reduces the space that a digital image occupies; it is a practical and efficient method for image compression that preserves sufficiently acceptable visual quality. It takes images, divides the images into blocks and quantizes frequency coefficients and thus removes data that s needed. It further lowers the demand in terms of storage and bandwidth. One can achieve a varying trade-off for the compression ratio and image quality by simply modifying their quantization settings.